# Practice Set 2 - NumPy Data Types & Functions

Use these exercises to practice NumPy concepts including data types, broadcasting, and mathematical functions.

## Section 1: Data Types in NumPy

1. Create an array `[1, 2, 3, 4, 5]` and check its `dtype` attribute.
2. Create an array `[10, 20, 30]` with `dtype='float32'` and print both the array and its dtype.
3. Create an integer array `[5, 10, 15]` and convert it to `float64` using `astype()`.
4. Create two arrays with the same values but different dtypes ( `int32` and `int64` ). Compare their `nbytes`.
5. Create an array with 100 elements of `float32`. Calculate how many bytes it uses and compare with `float64`.
6. Create an array from `[1.5, 2.5, 3.5]` as `int32`. What values do you get and why?
7. Create a float array `[1.9, 2.1, 3.7, 4.3]` and convert to `int32`. What happens to the decimal parts?
8. Create 4 different arrays with values `[10, 20, 30]` using dtypes: `int8`, `int32`, `float32`, `float64`. Print dtype and nbytes for each.

## Section 2: Broadcasting

9. Create array `[1, 2, 3, 4, 5]` and add 10 to each element using broadcasting.
10. Subtract a 1D array `[1, 2, 3]` from a 2D array `[[10, 20, 30], [40, 50, 60]]`.
11. Create a 3x3 matrix `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` and multiply every element by 5 using broadcasting.
12. Create a dataset and normalize it using broadcasting: subtract the column mean and divide by column standard deviation.
13. Create a 2x4 array and a 1D array with 4 elements. Add them together using broadcasting.
14. Create a column vector with shape `(3, 1)` and a row vector with shape `(1, 4)`. Multiply them and print the result shape.
15. Create a random dataset with shape `(5, 3)`. Center the data by subtracting the mean of each column.

# Section 3: Built-in Mathematical Functions

16. For array `[10, 20, 30, 40, 50]` , calculate mean, median, and standard deviation using `np.mean()` , `np.median()` , and `np.std()` .
17. For array `[5, 2, 8, 1, 9, 3]` , find the minimum, maximum, and range (max - min).
18. For array `[3, 5, 2, 33, 2, 3, 4, 3]` , use `np.argmin()` and `np.argmax()` to find indices and values of min/max.
19. For array `[1, 2, 3, 4, 5]` , calculate the sum using `np.sum()` and the product using `np.prod()` .
20. For array `[1, 2, 3, 4, 5]` , calculate the cumulative sum using `np.cumsum()` and understand the running total.
21. For array `[1, 2, 2, 3, 3, 3, 4, 4, 4, 4]` , use `np.unique()` with `return_counts=True` to find unique elements and their counts.
22. For array `[10, 15, 20, 25, 30]` , calculate variance using `np.var()` and verify it equals `(std)^2` .
23. For array `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` , find the 25th, 50th, and 75th percentiles using `np.percentile()` .
24. Create a 3x4 array and calculate mean along `axis=0` (columns), `axis=1` (rows), and overall mean.
25. For array `[1.1, 2.5, 3.9, 4.2, 5.8]` , round to nearest integer, floor, and ceiling using `np.round()` , `np.floor()` , `np.ceil()` .
26. For array `[-5, -2, 0, 3, -8, 7]` , calculate absolute values using `np.abs()` .
27. For array `[1, 4, 9, 16, 25]` , calculate square root, square, and cube root using `np.sqrt()` , `**2` , and `**(1/3)` .
28. For array `[0, np.pi/2, np.pi]` , calculate sine, cosine, and tangent using `np.sin()` , `np.cos()` , `np.tan()` .
29. For array `[5, 2, 8, 1, 9, 3]` , sort the array and get the indices that would sort it using `np.sort()` and `np.argsort()` .
30. For array `[10, 20, 30, 40, 50]` , calculate the coefficient of variation: `(std / mean) * 100` .